TRAINER LAB MANUAL | ELASTIC STACK 9.x

Observability Dashboards and Alerts

Step-by-step Kibana Lens + alert rules, expected output screens, Dev Tools checks and lab evidence.

- Self-managed Ubuntu / KVM Elasticsearch cluster
- Three ES nodes + Kibana + Fleet System integration
- Metricbeat X-Pack Stack Monitoring on node03
- L5 / L6 / Q3 training exercises

Screenshot integrity All included UI screenshots are illustrative training mock-ups, not captures of your Kibana. Actual outputs must be collected from your environment and submitted as evidence.

Prepared 04 November 2026 | Lab edition: 1.0 | Suggested duration: 3-4 hours

How to use this manual

Complete the read-only validation first. Build live host dashboards from Fleet System integration data. Build error dashboards from the synthetic Q3 dataset. Finally create and test alert rules using an isolated training index. Each lab has an expected outcome and evidence item.

Section	Objective / Evidence
0 - Data readiness	Verify CPU/memory documents and Kibana data views
1 - Dashboard design	Six Lens panels, saved dashboard, filter controls
2 - Stack Monitoring	Verify Metricbeat X-Pack node monitoring
3 - Alert rule design	CPU, memory, ERROR-count rules with appropriate thresholds
4 - Alert testing	Controlled synthetic data seed, fired/recovered states
5 - Trainer handover	Screenshots, rule health, incident note and cleanup

Target topology (verify the live VM addresses)

Host	Lab address / role
node01	192.168.122.227 - Elasticsearch HTTPS 9200
node02	192.168.122.246 - Elasticsearch HTTPS 9200
node03	192.168.122.102 - ES HTTPS 9200; Metricbeat installed
Kibana VM	192.168.1.162 - Kibana UI port 5601
Fleet Server	rps1 - address must be checked; Fleet Server port 8220

Critical separation Fleet System sends host CPU/memory to metrics-system.cpu-* and metrics-system.memory-*. Metricbeat X-Pack sends Elasticsearch component monitoring into Stack Monitoring. Do not treat them as interchangeable.

Before taking evidence

- Use the actual host.name value returned by Elasticsearch (node03 is an example).
- Keep the dashboard time picker at Last 30 minutes after ingestion.
- Use safe synthetic data for test alerts; do not trigger login failures or unbounded CPU load.
- Do not share passwords, enrolment tokens, HTTP credentials or sensitive log lines in screenshots.

0.1 Readiness - verify host metric data in Dev Tools

In Kibana, open Dev Tools > Console. Execute each request separately, using the play button. The data stream is confirmed in your earlier screenshots, but you must confirm that recent documents exist.

```
GET _data_stream/metrics-system.cpu-*
GET _data_stream/metrics-system.memory-*
GET metrics-system.cpu-*/_count
GET metrics-system.memory-*/_count
```

Check latest CPU sample and hostname:

```
GET metrics-system.cpu-*/_search
{
  "size": 3,
  "sort": [{"@timestamp": {"order": "desc"}}],
  "_source": ["@timestamp", "host.name",
              "system.cpu.total.norm.pct", "data_stream.dataset"]
}
```

Check memory independently; it is a different dataset:

```
GET metrics-system.memory-*/_search
{
  "size": 3,
  "sort": [{"@timestamp": {"order": "desc"}}],
  "_source": ["@timestamp", "host.name",
              "system.memory.actual.used.pct"]
}
```

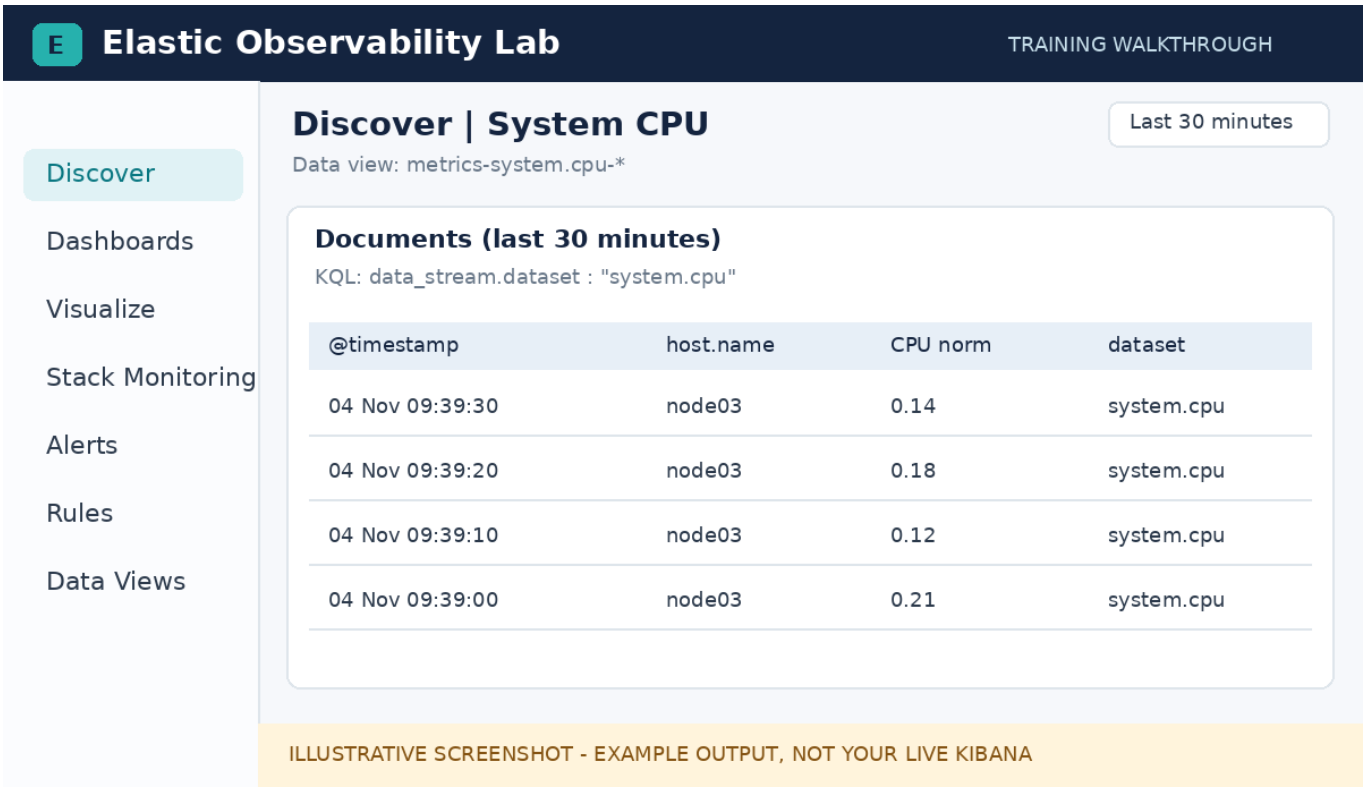
What good looks like A recent hits.hits array with _source.@timestamp, _source.host.name and the relevant normalized fraction. Example 0.25 = 25%, not 0.25%. The _data_stream API checks stream metadata; search documents with GET metrics-system.cpu-*/_search - never append /_search after /_data_stream/.

0.2 Prepare the four data views

Open Stack Management > Data Views > Create data view. Depending on privileges, use Kibana global search for Data Views. Choose @timestamp as the time field.

Data view name	Pattern	Use
L5 - CPU Metrics	metrics-system.cpu-*	Host CPU trend / CPU alert
L5 - Memory Metrics	metrics-system.memory-*	Host memory trend / memory alert
L5 - All Host Metrics	metrics-system.*	Other System datasets; use dataset filter
L6-Q3 - Synthetic Incidents	lab-incident-events	Error trends, noisy hosts

1. Select Create data view, type name and exact index pattern, choose @timestamp, and save.
2. Open Discover. Choose L5 - CPU Metrics, set Last 30 minutes.
3. Enter KQL: `data_stream.dataset : "system.cpu"` and optionally `host.name : "node03"`.
4. Add columns @timestamp, host.name, system.cpu.total.norm.pct.
5. Repeat with L5 - Memory Metrics and system.memory.actual.used.pct.



Expected output reference: CPU documents visible in Discover with timestamps and exact host name. Evidence: D0. Illustrative mock-up only; your real hostname, values, date and UI may differ.

0.3 Prepare the Q3 incident data for Lens

Your supplied L5/L6/Q3 manual defines a synthetic index named lab-incident-events (285 sample documents). Use that existing dataset, or rerun its generator when the documents fall outside the dashboard time window.

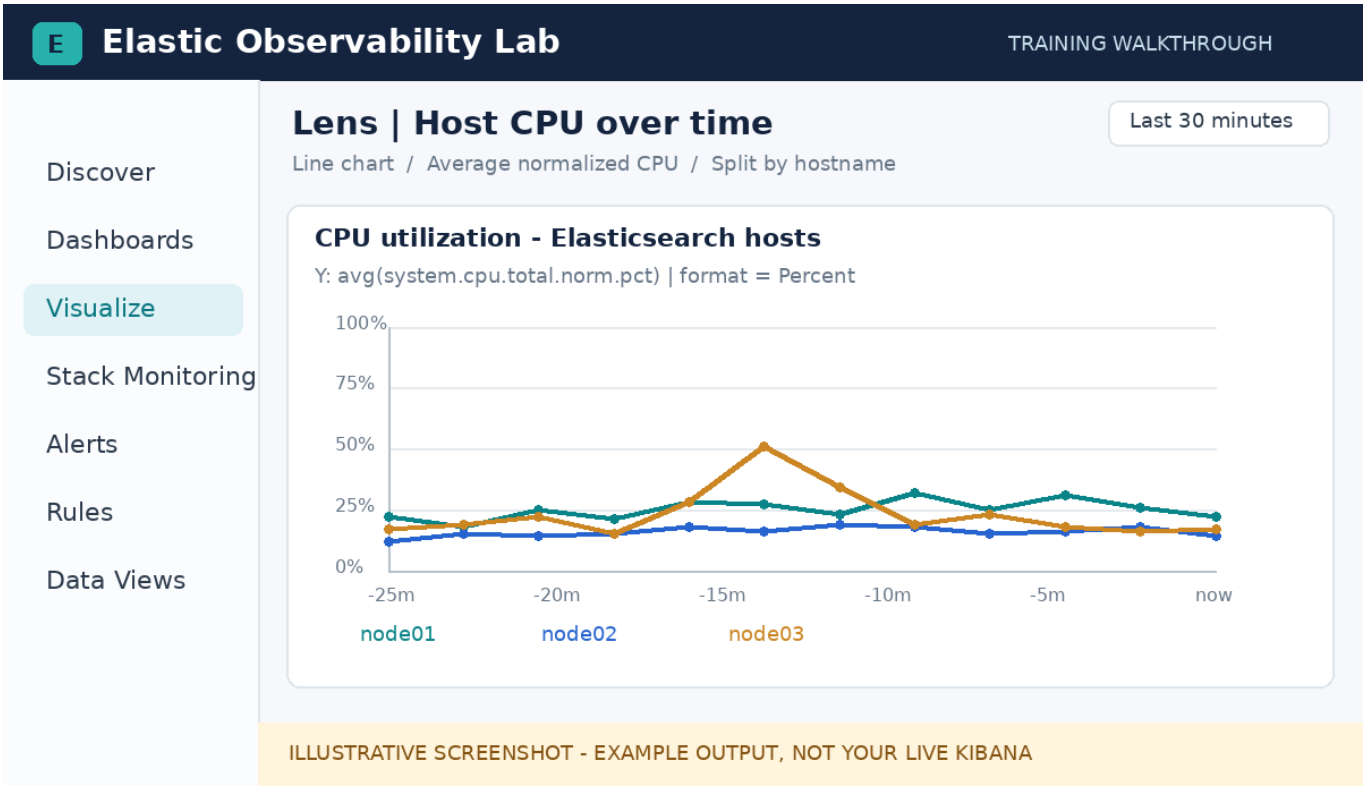
```
GET lab-incident-events/_count
GET lab-incident-events/_search
{
  "size": 2,
  "sort": [{"@timestamp": {"order": "desc"}}],
  "_source": ["@timestamp", "service.name", "host.name",
              "log.level", "lab.source_type", "message"]
}
```

1. If count is zero or the index does not exist, complete L6.5 from the original L5/L6/Q3 lab manual to load the synthetic sample. Do not mistake these for live server incidents.
2. Create the data view L6-Q3 - Synthetic Incidents with pattern lab-incident-events and timestamp @timestamp.
3. In Discover set Last 30 minutes. KQL: `lab.case_id : "L6-Q3-TRAINING"`.
4. If no results, inspect the actual newest @timestamp and regenerate the source (the sample uses now-relative timestamps).

> Expected sample distribution For the original pristine dataset, the sample totals 285 documents. Illustrative service/host ERROR counts are payments/app-linux-01 = 90, ingest-pipeline/logstash-lab-01 = 22, windows-audit/win-lab-01 = 18 and orders/app-linux-02 = 12. These are not live events.

1.1 Dashboard panel D1 - CPU line chart

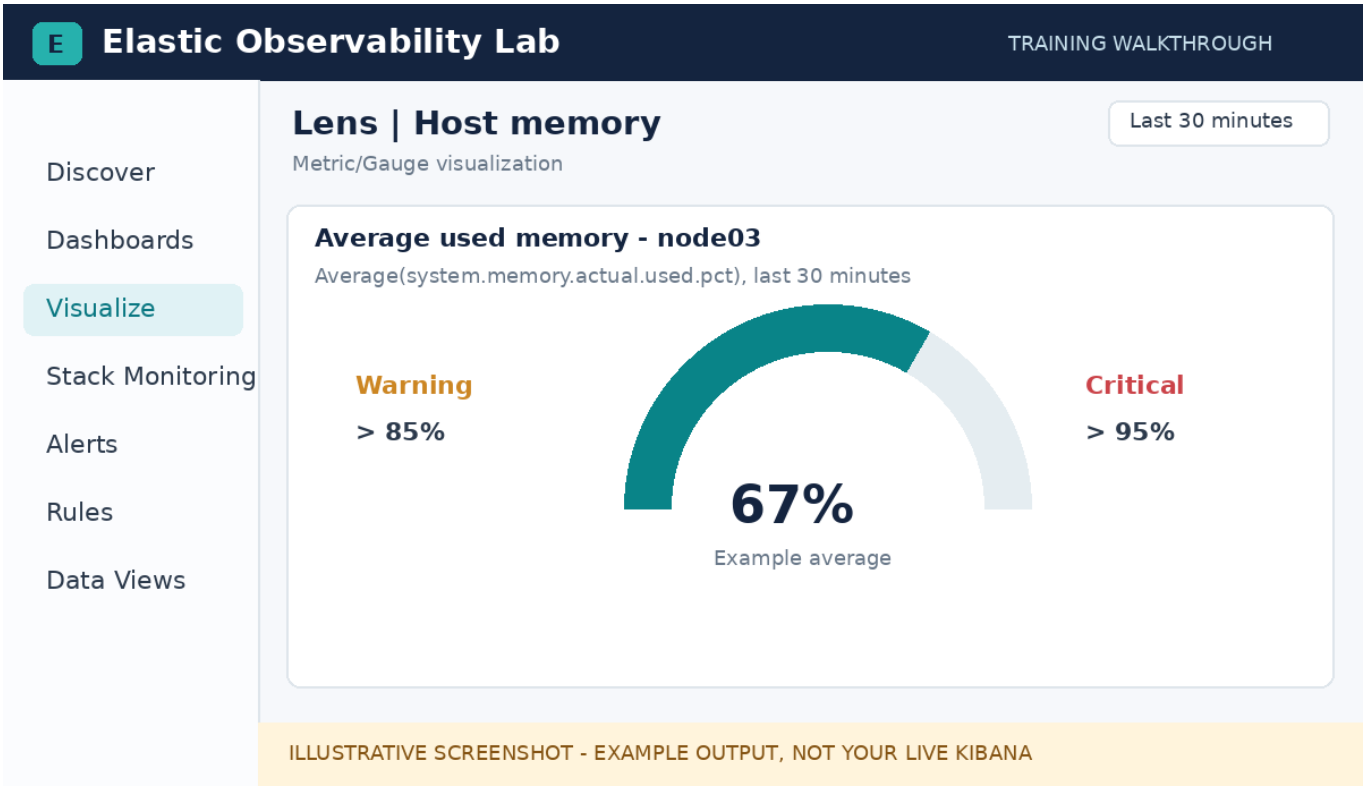
1. Go to Dashboards > Create dashboard > Add visualization (Lens).
2. Select data view L5 - CPU Metrics. Choose Line as visualization type.
3. Set horizontal axis to @timestamp (Date histogram / automatic interval).
4. Set vertical axis to Average of system.cpu.total.norm.pct. Change value format to Percent; underlying data are 0-1 fractions.
5. Optional: use Top values of host.name for Break down. For a node03-only dashboard apply KQL `host.name : "node03"`.
6. Title D1 - Host CPU % over time (LIVE). Select Save and return or Save to library.



Expected output reference: Three illustrative series; a real node03-only deployment may show only one series. Evidence: D1. Illustrative mock-up only; your real hostname, values, date and UI may differ.

1.2 Dashboard panel D2 - memory utilization

1. In the dashboard, select Add visualization. Set data view L5 - Memory Metrics.
2. Choose Metric or Gauge (depending on available Lens choices).
3. Metric: Average of system.memory.actual.used.pct. Format as Percent. Name it D2 - Avg host memory used % (LIVE).
4. Filter to host.name : "node03". If using Gauge, use range 0-1 and a reference/goal such as 0.85 (85%).
5. Important: a 30-minute average is not an instant measurement. Name it Avg/last 30m. For closer-to-current behavior use Last 5 minutes.
6. Save and return. Compare displayed values with one recent Discover memory document.



Expected output reference: Example 67% memory metric; real values depend on your host. Evidence: D2. Illustrative mock-up only; your real hostname, values, date and UI may differ.

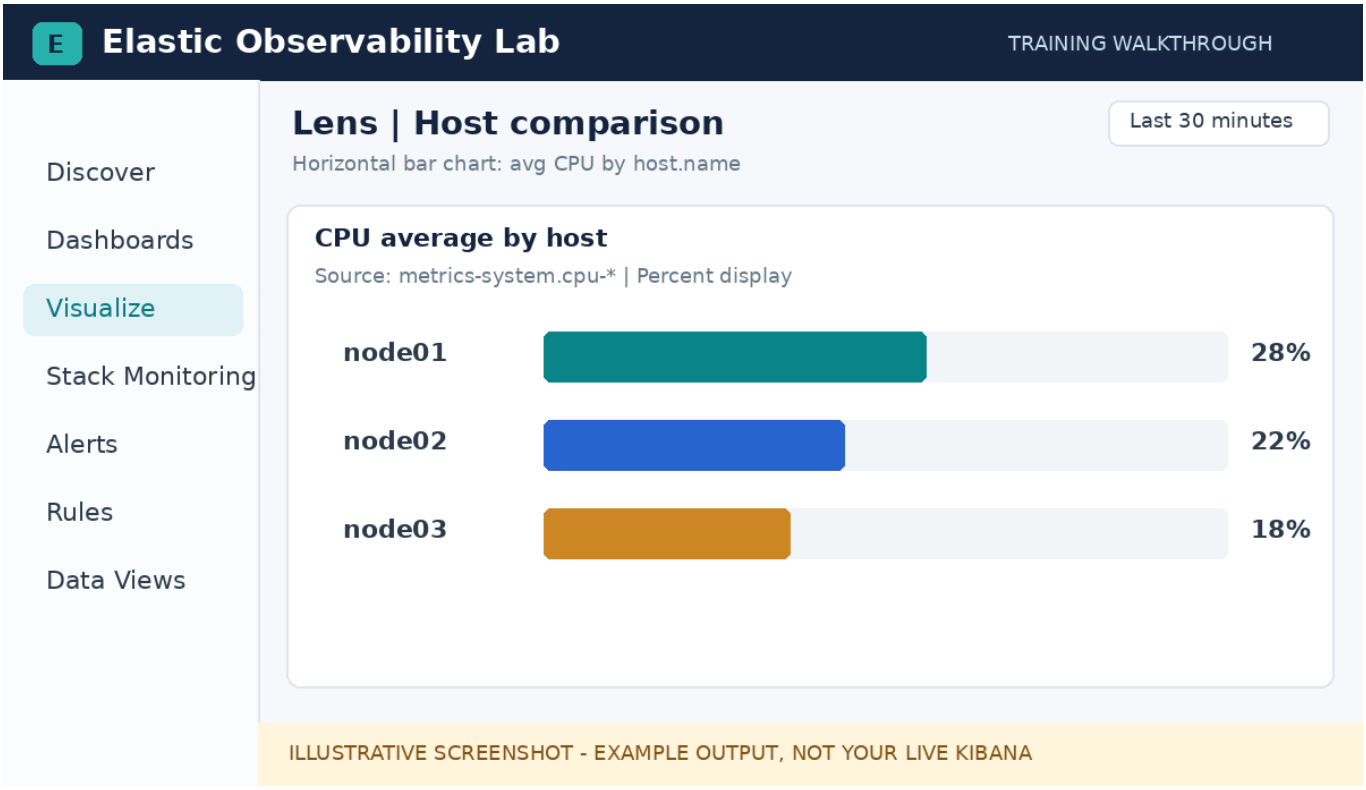
1.3 Dashboard panels D3/D4 - host comparison and disk

D3: Top host CPU (horizontal bar)

1. New Lens visualization; select L5 - CPU Metrics, chart Bar horizontal.
2. Category axis: Top values host.name. Metric: Average system.cpu.total.norm.pct.
3. Set number of host values = 10, descending by average. Format metric Percent and title D3 - Avg CPU by host (LIVE).

D4: Disk usage

4. Create a Lens bar/metric with data view L5 - All Host Metrics. Apply KQL: `data_stream.dataset : "system.filesystem"`.
5. Use Average system.filesystem.used.pct; if the field is absent, first verify the integration enabled filesystem metrics. Break down by host.name or system.filesystem.mount_point where present.
6. Title D4 - Filesystem usage % (LIVE). Limit to relevant mounted volumes; avoid averaging all mount points without explanation.



Expected output reference: Example only. If only node03 reports metrics, the comparison chart shows just node03. Evidence: D3/D4. Illustrative mock-up only; your real hostname, values, date and UI may differ.

1.4 Dashboard panel D5 - ERROR counts per minute

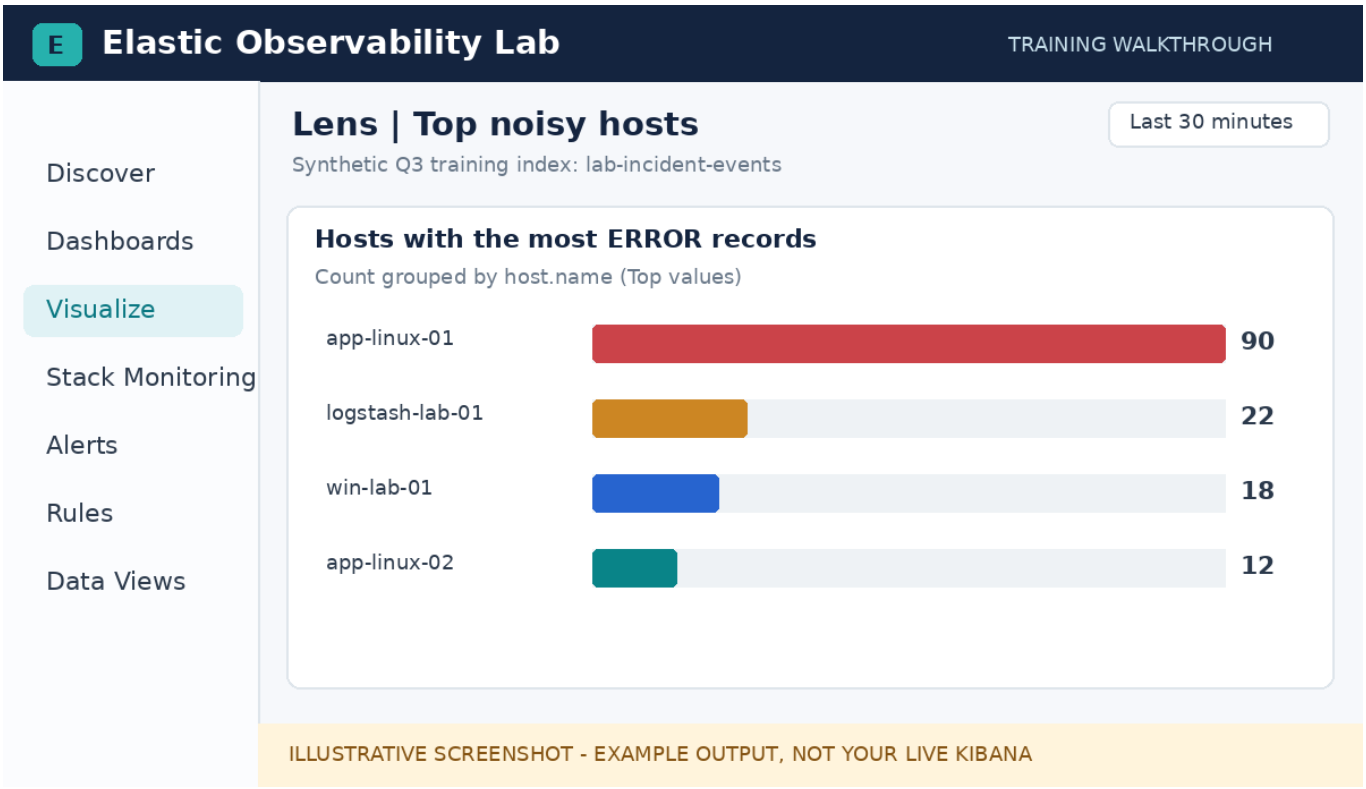
1. Add Lens visualization using L6-Q3 - Synthetic Incidents data view.
2. Set filter `lab.case_id : "L6-Q3-TRAINING" and log.level : "ERROR"`.
3. Choose Line (or stacked Bar). X-axis = @timestamp Date histogram; set 1 minute if the 30-minute window supports it.
4. Y-axis = Count of records. Break down = Top values service.name.
5. Title D5 - Errors by service/minute (SIMULATED) and save to dashboard.
6. Hover individual points to compare payment service errors with other services; take a screenshot showing breakdown legend.



Expected output reference: Synthetic error series. These counts prove chart configuration, not a real outage. Evidence: D5. Illustrative mock-up only; your real hostname, values, date and UI may differ.

1.5 Dashboard panel D6 - Top noisy hosts

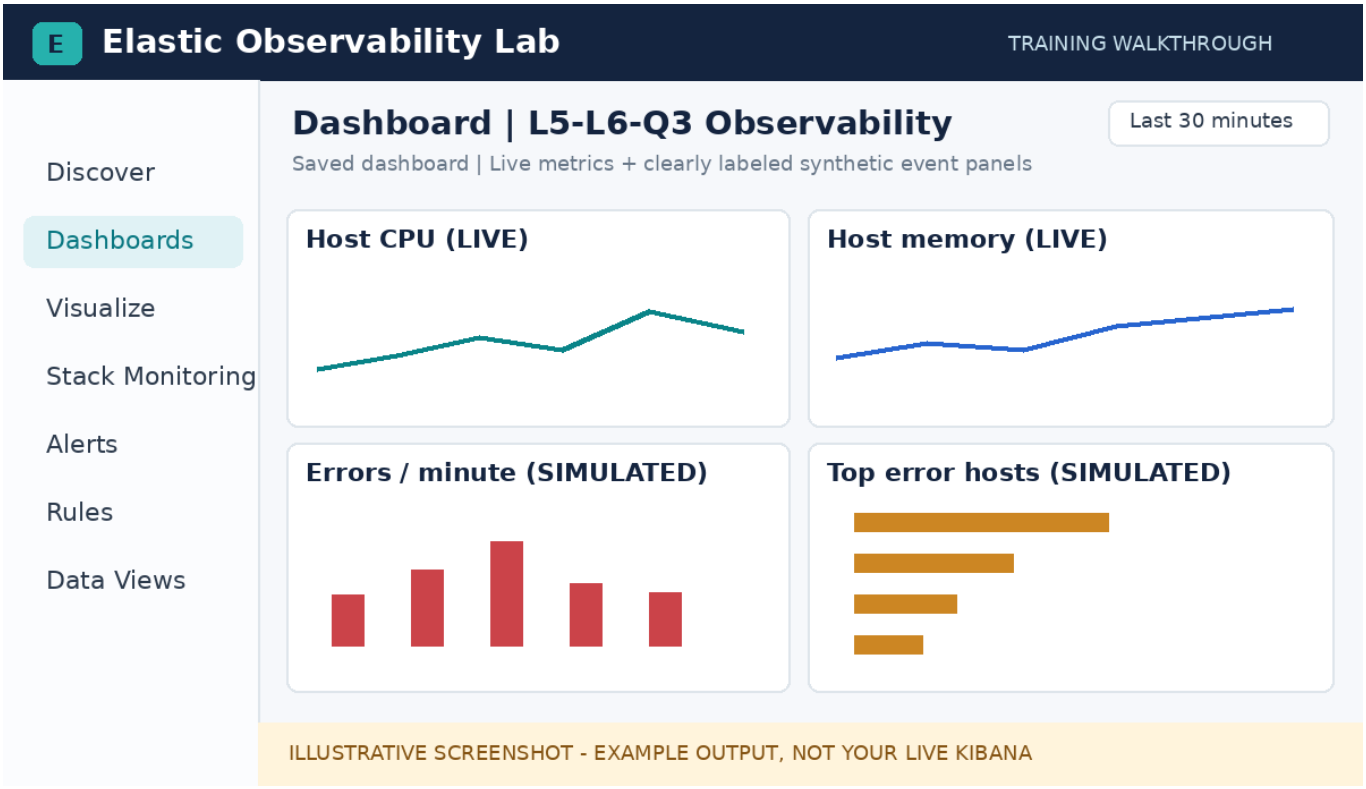
1. Add Lens visualization using L6-Q3 - Synthetic Incidents.
2. Filter `lab.case_id : "L6-Q3-TRAINING" and log.level : "ERROR"`.
3. Choose Horizontal bar. Category/Y = Top values host.name (up to 10). Value/X = Count of records.
4. Sort descending by count. Title D6 - Top hosts by ERROR count (SIMULATED).
5. Check that expected synthetic ranking is led by app-linux-01 when the original dataset is pristine.



Expected output reference: The illustrated 90/22/18/12 values come from the synthetic training dataset. Evidence: D6. Illustrative mock-up only; your real hostname, values, date and UI may differ.

1.6 Assemble, save and verify the dashboard

1. Go to Dashboards and open your working dashboard in Edit mode.
2. Place D1 CPU and D2 memory in the first row; D3 host CPU and D4 disk in the second; D5 errors and D6 noisy hosts in the third. Resize for readability.
3. Add an Options list control for host.name only if the panels share compatible data views. A dashboard-wide filter can inadvertently empty your synthetic panels (different hosts).
4. For a mixed dashboard, filter hosts inside each live-metric panel; use separate synthetic filtering in each log panel. Add text labels LIVE and SIMULATED.
5. Choose Save. Title: L5-L6-Q3 | Ubuntu Observability Dashboard. Save time Last 30 minutes if offered.
6. Reopen in view mode and confirm all panels draw. Refresh when the input data or time window changes.



Expected output reference: Example composition. Add the D3/D4 panels as space allows. Evidence: D7 (whole dashboard). Illustrative mock-up only; your real hostname, values, date and UI may differ.

1.7 Dashboard quality checks, controls and export

Five acceptance tests

- Time: Last 30 minutes covers recent metrics and newly generated synthetic logs.
- Fields: CPU and memory fractions are formatted as percent; no multiplying twice by 100.
- Legend: all charts explain which node/service they represent.
- Provenance: every training-data panel displays SIMULATED in its title.
- Persistence: Saved dashboard reopens with its panels and acceptable filters.

Export and trainer evidence

1. In dashboard view choose Share / Export if available. CSV export is available for underlying tabular data in suitable contexts; screenshot capture works without a reporting license.
2. Export saved objects as JSON using the Saved Objects management feature if your role and UI permit; confirm linked data views and visualizations are included as needed.
3. Capture full-screen dashboard screenshot with page title and time window. Keep actual charts separate from the illustrated figures in this manual.

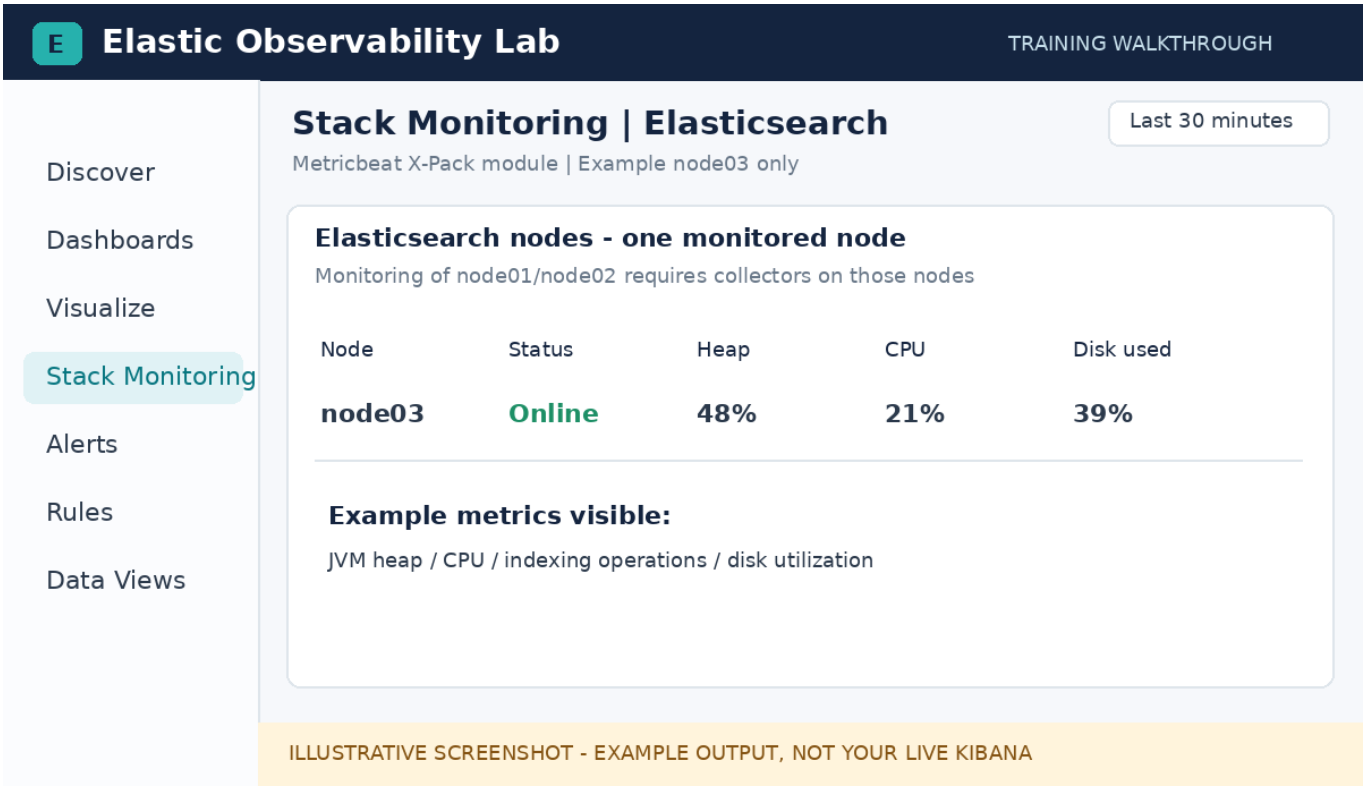
Panel troubleshooting If only CPU appears, memory may be in metrics-system.memory-*, not the CPU data view. If D5/D6 disappear after 30 minutes, regenerate the synthetic dataset using its original L6.5 script and refresh.

2.1 Elasticsearch Stack Monitoring - Metricbeat

This section checks your separate Metricbeat 9.5.6 Elasticsearch X-Pack module. It is not the input source for Lens CPU panels built from Fleet System integration.

1. On node03 verify sudo systemctl status metricbeat --no-pager reports active (running).
2. Verify sudo metricbeat modules list shows elasticsearch-xpack enabled and the regular elasticsearch module disabled.
3. Verify sudo metricbeat test output -e and sudo metricbeat test modules elasticsearch -e. If output fails, fix keystore credentials, HTTPS URL and CA.
4. In Kibana navigate Stack Monitoring > Elasticsearch > Nodes. Set Last 30 minutes.
5. Inspect node03 JVM heap, CPU, disk and indexing metrics. When only node03 is monitored, evidence is partial; add Metricbeat collectors to node01 and node02 for complete node-level coverage.

```
GET _cat/indices/.monitoring*?v&expand_wildcards=all
GET _data_stream/.monitoring*?expand_wildcards=all
```



Expected output reference: One example monitored node. Evidence: E4 from the original L5.7 lab. Illustrative mock-up only; your real hostname, values, date and UI may differ.

3.1 Alerting prerequisites and rule selection

Use Kibana Observability > Alerts > Manage rules or Stack Management > Rules, depending on your navigation. The exact control labels depend on the version, permissions and license.

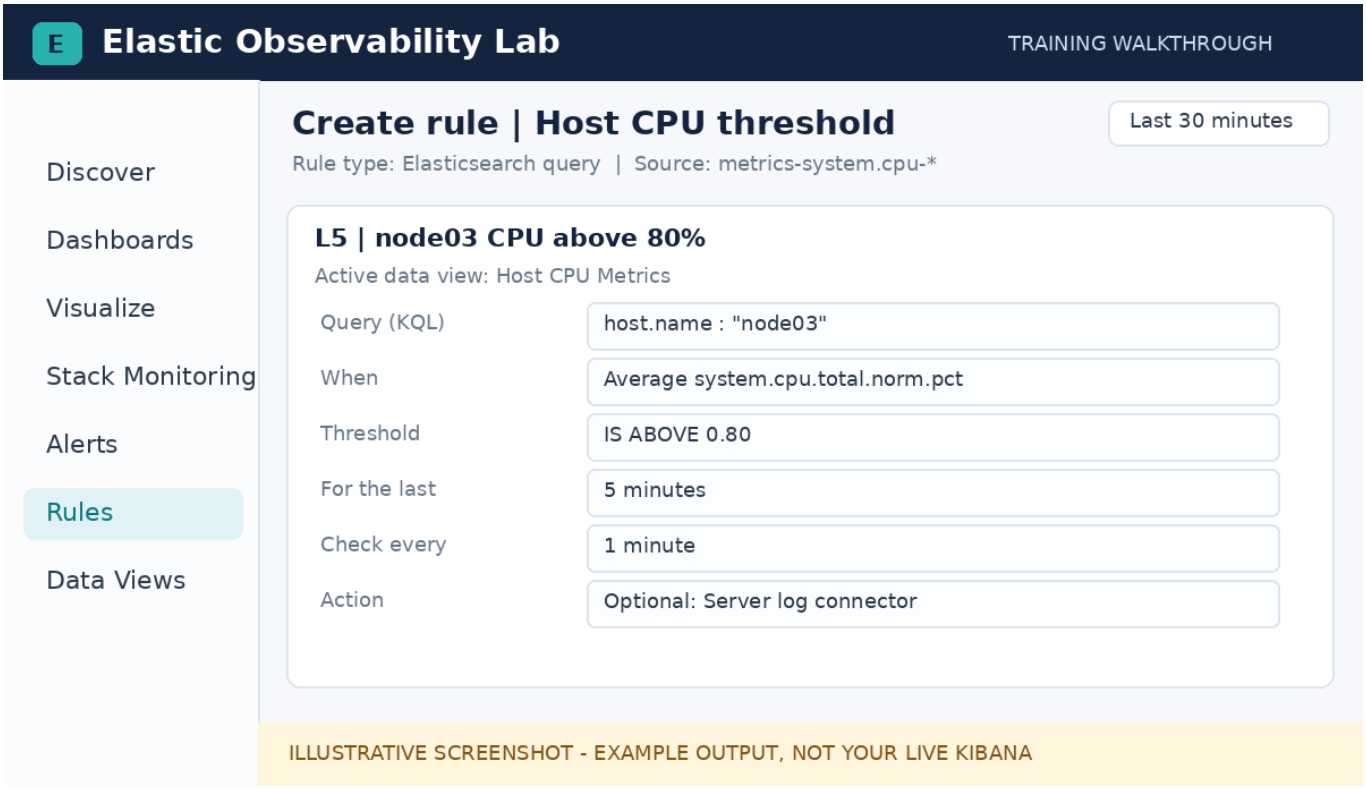
1. Confirm your Kibana role can create rules and read the required metric/log indices. Alerts are evaluated with a rule API key associated with the saving user.
2. On self-managed Kibana, ensure a persistent xpack.encryptedSavedObjects.encryptionKey is configured. Do not rotate an existing working key for this exercise.
3. Open Rules > Create rule. For deterministic training, use the broadly applicable Elasticsearch query rule type. Metric threshold or Custom threshold is an optional alternate if visible.
4. Choose the correct data view and @timestamp. The schedule is normally every 1 minute for this lab, with evaluation over the preceding 5 minutes.
5. Actions use connectors and can require additional configuration/license. The Kibana Alerts list can still be used to inspect rule outcomes even if external notification actions are unavailable.

Rule	Source	Condition / Purpose
A1 Host CPU	metrics-system.cpu-*	Average fraction > 0.80 over last 5m
A2 Host memory	metrics-system.memory-*	Average fraction > 0.85 over last 5m
A3 Error spike	lab-observability-alert-events	Count > 3 ERROR events in 5m; testable
A4 Missing data (optional)	metrics-system.cpu-*	Ungrouped count < 1 for node03 / 10m

Threshold semantics A normalized system CPU value 0.80 means 80%. The rule value is numeric, not a formatted percent string. Use field-level verification in Discover before creating rules.

3.2 Alert rule A1 - node03 high CPU

1. Open Rules > Create rule; title L5 | node03 CPU above 80%. Select rule type Elasticsearch query.
2. Query type KQL. Choose data view L5 - CPU Metrics. Enter `host.name : "node03"` (replace with your actual indexed host.name).
3. Set When = Average, field system.cpu.total.norm.pct; choose Is above 0.80.
4. Set For the last = 5 minutes; check every 1 minute. Use ungrouped test for node03 or group by host.name if monitoring many hosts.
5. Click Test query if available and confirm it returns data. Optionally add a Server log action; save the rule.
6. Expect status OK when below threshold; a saved healthy rule is evidence even if no real CPU breach occurs.



Expected output reference: Example configuration fields; CPU rule does not have to fire during this lab. Evidence: A1. Illustrative mock-up only; your real hostname, values, date and UI may differ.

3.3 Alert rule A2 - high host memory

1. Create rule L5 | node03 memory above 85% of type Elasticsearch query.
2. Select KQL and L5 - Memory Metrics data view.
3. Filter `host.name : "node03"` (replace with actual hostname).
4. When = Average of system.memory.actual.used.pct. Threshold: Is above 0.85. Last 5 minutes; run every 1 minute.
5. Use Test query. Save rule. Review Last response, Rule health, Execution history where present.
6. Do not deliberately consume RAM to force an alert on a shared ES VM; use the synthetic A3 exercise to prove action/alert flow.

```
GET metrics-system.memory-*/_search
{
  "size": 3,
  "query": {"term": {"host.name": "node03"}},
  "sort": [{"@timestamp": "desc"}],
  "_source": ["@timestamp", "host.name",
              "system.memory.actual.used.pct"]
}
```

Optional Metric threshold UI If your Kibana exposes Infrastructure > Alerts > Metric threshold, you can define average CPU/memory metrics and group alerts by host.name. That rule uses Infrastructure metric source settings (default metrics-*,metricbeat-*) rather than an arbitrary per-rule index pattern.

4.1 Safe alert test - create a dedicated synthetic index

This lab produces tiny synthetic ERROR documents with a server-generated ingestion timestamp. They do not represent actual failed services. Run in Kibana Dev Tools, not an Ubuntu shell. Your account needs pipeline/index creation and write privileges.

Step A - add timestamp automatically

```
PUT _ingest/pipeline/lab-observability-stamp
{
  "description": "Timestamp synthetic training events at ingest",
  "processors": [
    {"set": {"field": "@timestamp",
             "value": "{{{_ingest.timestamp}}}"}}
  ]
}
```

Step B - create a lab-only index using that pipeline

```
PUT lab-observability-alert-events
{
  "settings": {
    "index.default_pipeline": "lab-observability-stamp"
  },
  "mappings": {"properties": {
    "@timestamp": {"type": "date"},
    "host": {"properties": {"name": {"type": "keyword"}}},
    "service": {"properties": {"name": {"type": "keyword"}}},
    "log": {"properties": {"level": {"type": "keyword"}}},
    "message": {"type": "text"}
  }}
}
```

Important If this index or pipeline already exists, inspect rather than blindly overwriting it. For a clean new run, choose an unused lab-specific name consistently in all subsequent steps.

4.2 Safe alert test - bulk load 8 ERROR documents

Paste this complete bulk request in Dev Tools. Bulk requests must have alternating action and document lines. The ingest pipeline fills @timestamp with the current ingestion time.

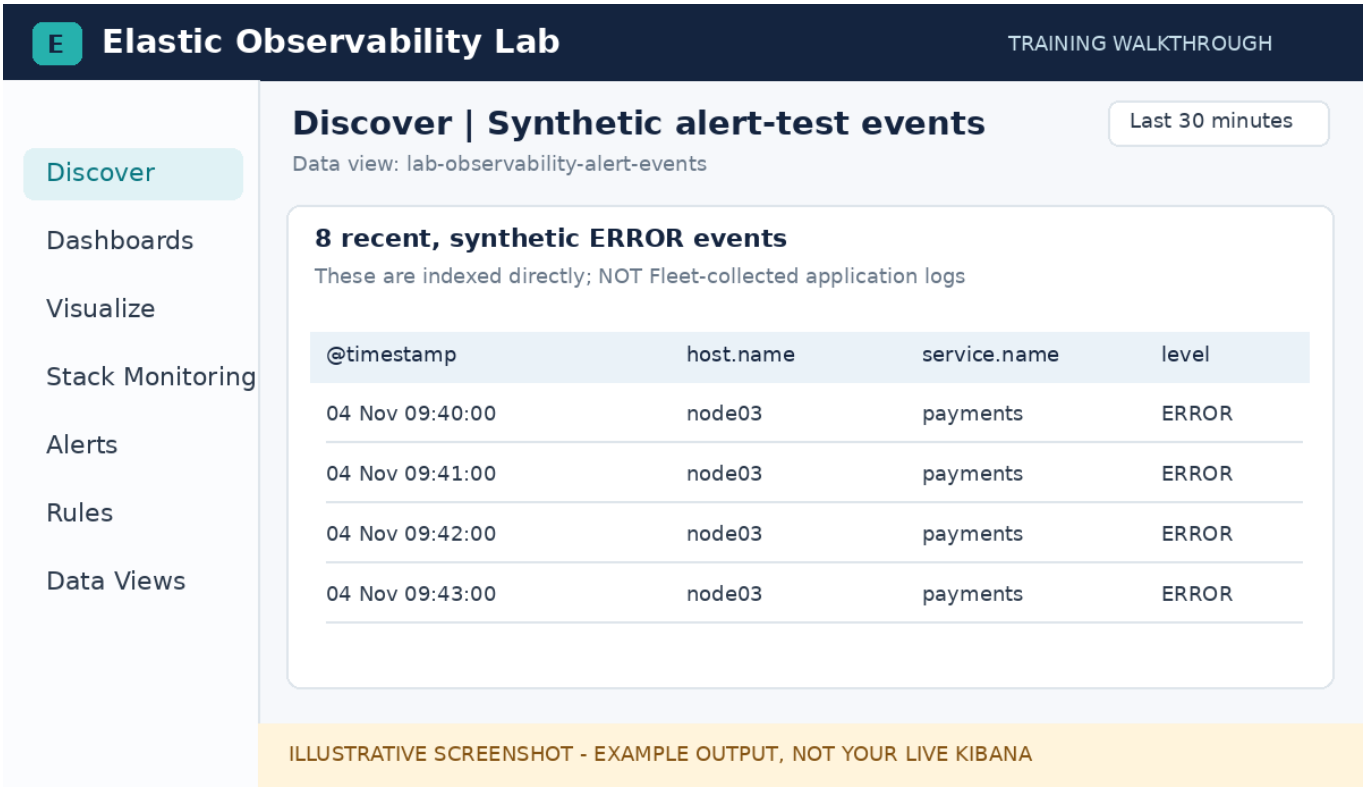
```
POST lab-observability-alert-events/_bulk?refresh=wait_for
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-001 synthetic training alert"}
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-002 synthetic training alert"}
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-003 synthetic training alert"}
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-004 synthetic training alert"}
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-005 synthetic training alert"}
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-006 synthetic training alert"}
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-007 synthetic training alert"}
{"index":{}}
{"host":{"name":"node03"},"service":{"name":"payments"},"log":{"level":"ERROR"},"message":"LAB-SIM-008 synthetic training alert"}
```

Expect the bulk response `"errors": false` and 8 successful items. Check individual item errors if errors=true.

4.3 Confirm synthetic documents in Discover

```
GET lab-observability-alert-events/_count
GET lab-observability-alert-events/_search
{
  "size": 10,
  "sort": [{"@timestamp": {"order": "desc"}}],
  "_source": ["@timestamp", "host.name", "service.name",
              "log.level", "message"]
}
```

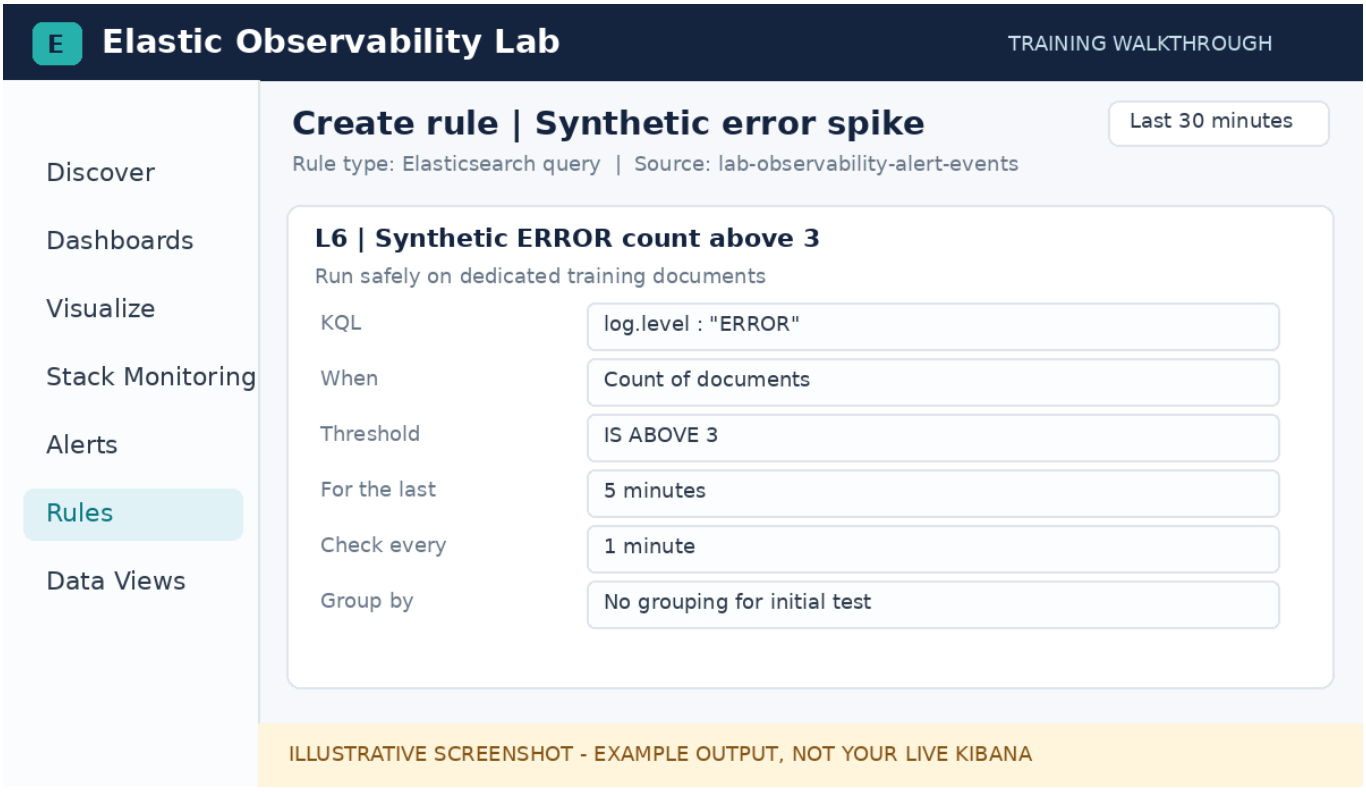
1. Create data view L6 - Alert Test Events using index lab-observability-alert-events, timestamp @timestamp.
2. Open Discover, choose data view L6 - Alert Test Events and time Last 15 minutes.
3. KQL: `log.level : "ERROR" and service.name : "payments"`. Expect at least 8 matching documents (more if you repeated bulk loads).
4. Verify field message begins with LAB-SIM so the training provenance is obvious.



Expected output reference: 8 simulated ERROR documents. Evidence: A3 test data, not live ingestion. Illustrative mock-up only; your real hostname, values, date and UI may differ.

4.4 Alert rule A3 - ERROR count spike

1. Create rule L6 | Synthetic ERROR count above 3; type Elasticsearch query.
2. Query type KQL. Select data view L6 - Alert Test Events. Filter `log.level : "ERROR"`.
3. When = Count of documents, is above 3, For last 5 minutes.
4. Check every 1 minute. For the first test do not group results; later use Grouped over service.name to see per-service alerts.
5. Use Test query. With 8 fresh documents it should display a count at least 8; if it does not, check timestamps and data view.
6. In some versions Exclude matches from previous run can suppress alerts for already seen documents. For a controlled repeating count-based lab, you may turn it off temporarily; return to your preferred deduplication policy afterwards.
7. Save, ensure rule enabled, and inspect its execution state after the first scheduled evaluation.

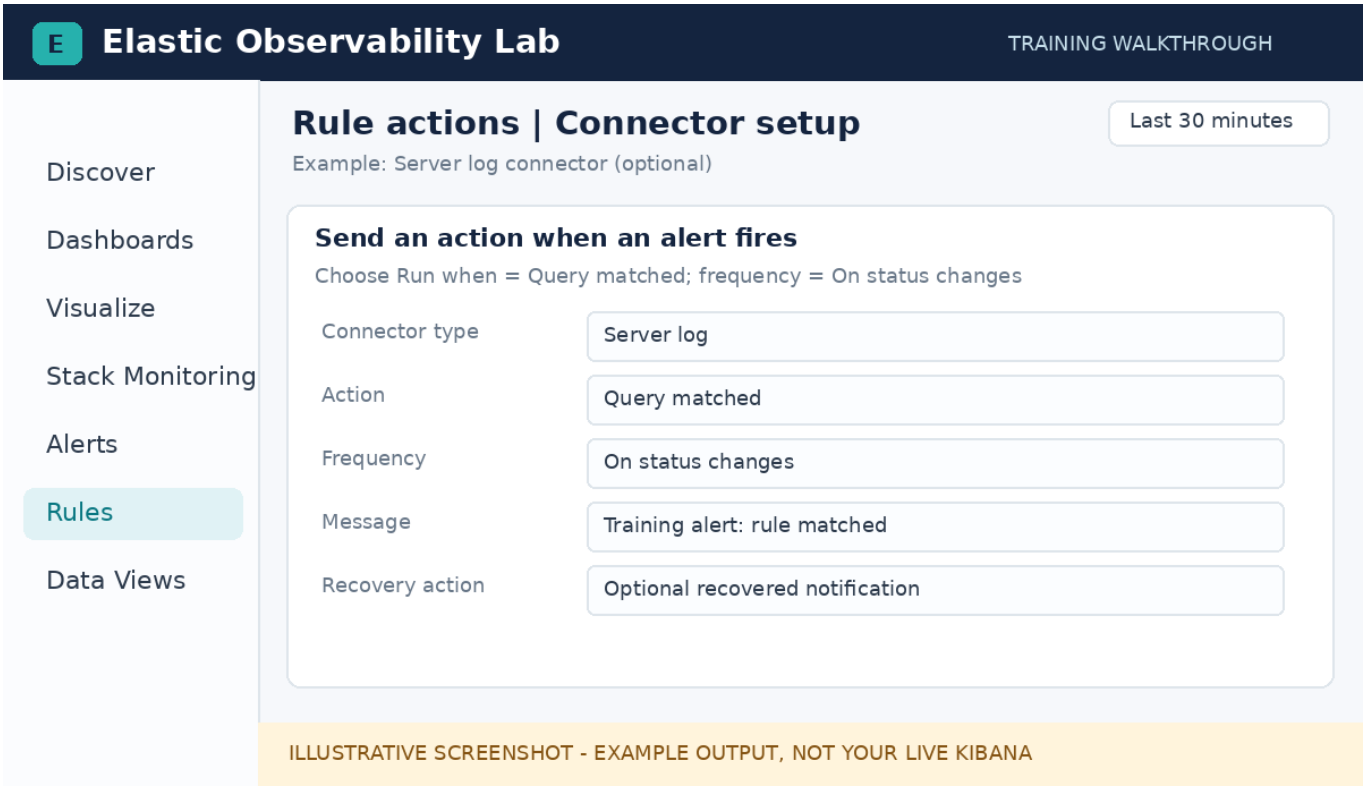


Expected output reference: Expected rule form; the actual trigger depends on indexing time and scheduler health.
Evidence: A3. Illustrative mock-up only; your real hostname, values, date and UI may differ.

4.5 Add an optional Server log action

Connectors are optional to demonstrate alert detection. Alert instances can be viewed in Kibana Alerts without setting up email or a webhook. For a low-risk training action, use the Server log connector when available.

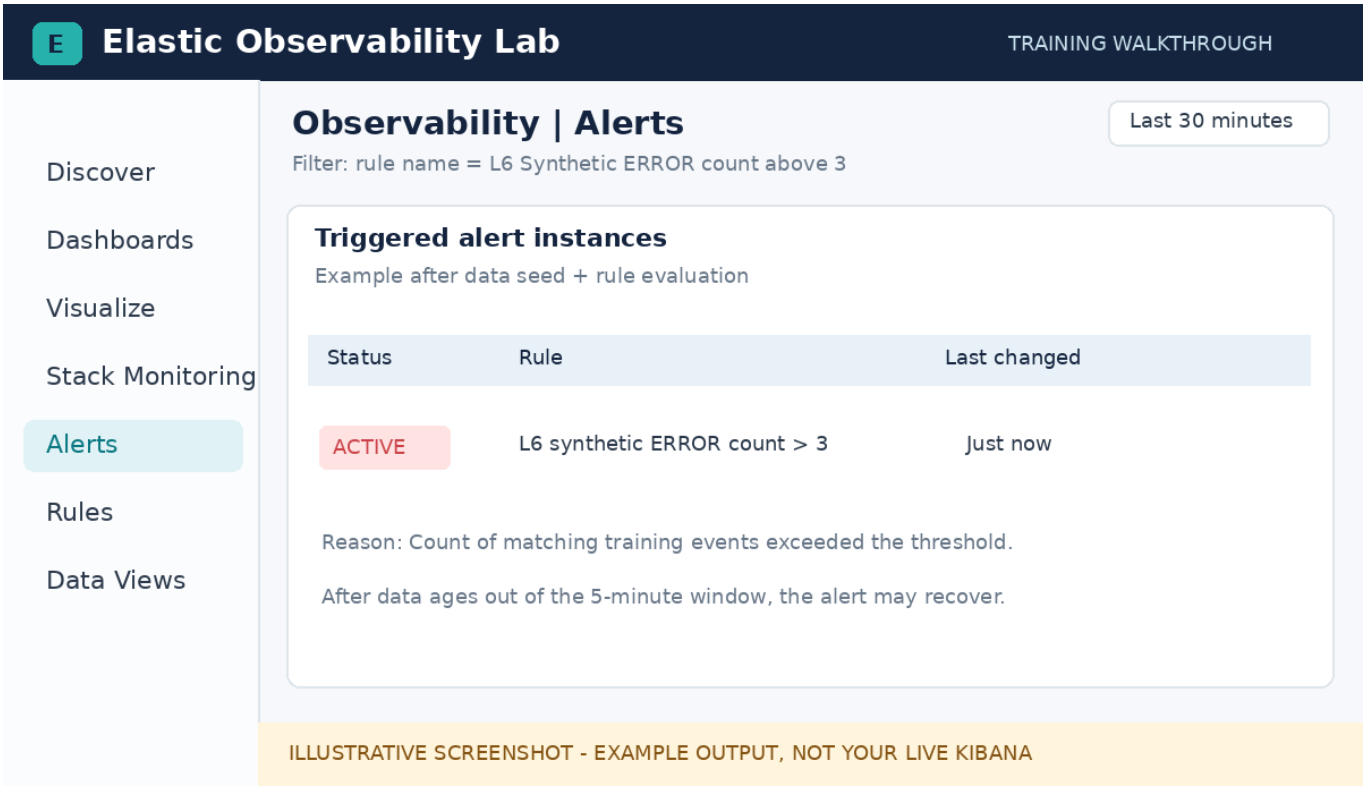
1. In rule editor find Actions > Add action. Select connector type Server log. Create it if authorized, or select an existing training connector.
2. Set action group Query matched (or relevant "Alert" group), and action frequency On status changes to limit repeated messages.
3. Use a short message such as L6 `test: {{rule.name}} matched (value {{context.value}})`. Choose rule variables from Kibana's variable picker if a token is not offered.
4. Optionally add an action when Recovered. Save the rule, then inspect Rule event log / Execution history and Kibana server logs if you have host access.
5. If you do not have a Server log connector or action rights, omit this step. Record the restriction; do not invent a sent notification.



Expected output reference: Illustrative action configuration. Action availability depends on your deployment. Evidence: A4. Illustrative mock-up only; your real hostname, values, date and UI may differ.

4.6 Verify ACTIVE alerts and rule health

1. Open Observability > Alerts (or Rules > selected rule > Alerts). Filter by L6 | Synthetic ERROR count above 3.
2. Inspect Active alert status and last change time. Open alert details to view the rule reason and metric/count.
3. If the alert is not present, check the rule is enabled, scheduled evaluations completed successfully, the 5-minute window contains the 8 documents, and the query test count exceeds 3.
4. When the event window passes, the rule should return to normal/recovered when its count drops below threshold, provided it continues evaluating successfully.
5. Capture two actual screenshots: rule details/health and Alerts table with timestamp/rule/status.



Expected output reference: Illustrative active alert after a successful scheduled evaluation. Evidence: A5. Illustrative mock-up only; your real hostname, values, date and UI may differ.

4.7 Optional A4 - missing CPU data alert

No-data alerts are especially important because a stopped agent may look like a perfectly calm CPU dashboard. Use the Elasticsearch query type with one fixed host and no grouping.

1. Name L5 | node03 CPU data missing 10m. Query KQL `host.name : "node03"`, data view L5 - CPU Metrics.
2. Set Count is below 1 for last 10 minutes, check every 1 minute. Do not group by host.name here; a missing host returns no bucket to evaluate.
3. Test query with current data. If the host is reporting, the rule should normally remain OK. A real outage is not required to complete this exercise.
4. For a real no-data alert, prefer an Infrastructure Metric threshold rule with its no data behavior when available; use the official UI to select recovery vs alerting on absent samples.
5. Do not stop Elasticsearch or Fleet Agent on a shared lab just to trigger the rule. Instead document how you would detect a gap from historical data.

Why grouping matters A grouped alert is only evaluated for groups the query returns. When a host completely stops reporting, it might not produce any bucket. For a fixed host no-data rule, filter to the host and evaluate an ungrouped count.

4.8 Verify recovery and avoid alert fatigue

Recovery demonstration

1. Wait until the synthetic events age out of the configured 5-minute lookback (without adding new events). Keep the rule enabled.
2. Review the Alerts table and rule history for the transition from Active to Recovered/Closed, depending on Kibana version.
3. If an alert persists, inspect alert instance behavior, overlapping evaluation windows, "Exclude matches" choice and timestamp freshness.

Operational controls

- Keep action frequency On status changes or choose a reasonable custom interval to avoid one message per run.
- Use host.name/service.name grouping only when it adds actionable context; cap high-cardinality groups.
- Record ownership, severity, threshold rationale, and runbook link in the rule description or tags.
- Set maintenance/snooze or disable test rules once evidence is collected; do not leave noisy experimental rules running.
- Do not configure real email/PagerDuty/webhook recipients for this lab without explicit authorization.

Good trainer evidence For a completed alert exercise, provide: configuration screenshot + rule enabled/healthy status + actual ACTIVE alert screenshot + recovery screenshot if observed, plus the exact data source and evaluation window.

5.1 ES|QL optional investigations

Run the following in Kibana Discover ES|QL mode against the training Q3 dataset. Use Last 30 minutes only when supported by your ES|QL UI; the queries shown below do not automatically include the Discover time filter in every invocation context.

Errors by service and minute

```
FROM lab-incident-events
| WHERE lab.case_id == "L6-Q3-TRAINING"
    AND log.level == "ERROR"
| EVAL minute = DATE_TRUNC(1 minute, @timestamp)
| STATS errors = COUNT(*) BY minute, service.name
| SORT minute ASC, errors DESC
| LIMIT 100
```

Noisiest hosts

```
FROM lab-incident-events
| WHERE lab.case_id == "L6-Q3-TRAINING"
    AND log.level == "ERROR"
| STATS errors = COUNT(*) BY host.name
| SORT errors DESC
| LIMIT 10
```

Host CPU trend (field extraction varies by dataset)

```
FROM metrics-system.cpu-*
| WHERE host.name == "node03"
| EVAL minute = DATE_TRUNC(1 minute, @timestamp)
| STATS avg_cpu = AVG(system.cpu.total.norm.pct) BY minute
| SORT minute ASC
| LIMIT 100
```

The host CPU field is a 0-1 fraction; multiply by 100 only if you want numeric percent output. For unsupported syntax or missing fields, prefer Lens / Dev Tools queries validated earlier.

5.2 Troubleshooting - no chart values

Symptom	Cause / next check
CPU chart empty	Check System integration / metrics-system.cpu-* / host.name / Last 30m.
Memory chart empty	Check metrics-system.memory-* separately and actual used field.
Dashboard shows 0.23%	Incorrect percent formatting; underlying 0.23 means 23%.
Q3 chart empty	Synthetic index exists? Sample timestamps now-relative? Correct data view?
CPU host breakdown empty	Only node03 reporting; do not fabricate node01/node02.
Disk gauge empty	System filesystem dataset disabled; verify system.filesystem.used.pct.
Metricbeat monitor blank	ES X-Pack module + output/TLS roles + Stack Monitoring time.

Useful Kibana Dev Tools checks

```
GET _cat/indices?v&s=index
GET _data_stream/metrics-system.*
GET metrics-system.cpu-*/_count
GET metrics-system.memory-*/_count
GET lab-incident-events/_count
GET lab-observability-alert-events/_count
```

Data streams versus indices For Fleet metrics, data streams may have hidden backing indices. Prefer querying GET _data_stream/metrics-system.* and search the data stream name directly. A plain _cat/indices without expand_wildcards may not show hidden backing indices.

5.3 Troubleshooting - rule does not trigger

Symptom	Cause / next check
Rule cannot be saved	Check Kibana alerting privileges/API keys/encryption key.
Rule execution failed	Rule details, execution log, read/index permissions, task scheduler.
Test query = 0	Data view + KQL + @timestamp + selected time window.
Synthetic events too old	Bulk seed again; the rule looks back only 5 minutes.
Count > 3 yet no alert	Is rule enabled? Has first check run? Dedupe behavior?
Action missing	Connector permissions, availability/license and action schedule.
Alerts appear repeatedly	Review alert frequency, grouping and duplicate-match exclusion.
Rule is OK but CPU chart high	Check exact host, average vs max, threshold and evaluation window.

Metricbeat / Fleet commands on node03

```
sudo systemctl status metricbeat --no-pager
sudo metricbeat modules list
sudo metricbeat test config -e
sudo metricbeat test output -e
sudo journalctl -u metricbeat -n 70 --no-pager
sudo systemctl status elastic-agent --no-pager
sudo elastic-agent status
```

Avoid enabling duplicate Elasticsearch monitoring on the same node through both Elastic Agent and Metricbeat. For L5.4 Fleet System metrics, an existing healthy Elastic Agent is sufficient.

5.4 Trainer evidence checklist and handover

ID	Required screenshot or artifact
D0	Discover shows real node03 CPU document @timestamp + host.name + CPU field
D1-D4	CPU trend, memory metric, host comparison, disk panel (live metrics)
D5-D6	Error timeline and top error hosts; labeled SIMULATED
D7	Saved dashboard reopened, all panels and Last 30 minutes visible
E4	Stack Monitoring evidence (Metricbeat X-Pack node03)
A1-A2	Saved host CPU and memory rule definitions; health/last run
A3	Seeded lab index + ERROR-count query/rule configuration
A4	Connector action details if connector authorized/available
A5	Actual Kibana alert instance; active and optionally recovered
Q3	ES|QL error/noisy-host result and brief incident explanation

Concise incident note template

- Incident/training case: L6-Q3 training (synthetic).
- Time window and timezone: record observed Kibana evaluation window and UTC timestamps.
- Observed evidence: actual error counts per service, top host, screenshots and ES|QL.
- Live correlation: host metrics collected by Fleet and node03 Stack Monitoring, if visible.
- Conclusion: distinguish hypothesis from proven cause; synthetic logs alone do not prove a real incident.
- Resolution: synthetic rule disabled or snoozed; lab data retention/cleanup choice recorded.

5.5 Clean up test assets safely

When students have captured evidence, disable the synthetic ERROR-count alert rule and remove any optional training connector actions if appropriate. Only delete the dedicated index/pipeline if you own them and no other exercise uses them.

Cleanup order

1. In Rules, disable L6 | Synthetic ERROR count above 3 to stop repeated evaluation and any notifications.
2. Export/capture the screenshots and any trainer results. Verify the saved dashboard no longer depends on the synthetic alert-test data unless intentional.
3. Optional, only for the dedicated assets created by this manual, run in Dev Tools:

```
DELETE lab-observability-alert-events
DELETE _ingest/pipeline/lab-observability-stamp
```

4. Leave metrics-system.*, .monitoring*, lab-incident-events, the System integration, and Metricbeat running unless your separate lab lifecycle requires otherwise.

> Safety guard Do not delete Elasticsearch system indices, real monitoring data streams or your previous labs. Delete only the exact lab-prefixed training index and ingest pipeline if explicitly intended.

Official references and source alignment

This manual expands the provided L5/L6/Q3 and Metricbeat training material with dashboard/alert design exercises. URLs below are official documentation used to verify Kibana workflows. UI labels may change slightly across patches, roles, spaces and license tiers.

Lens visualizations
https://www.elastic.co/docs/explore-analyze/visualize/lens

Create a dashboard
https://www.elastic.co/docs/explore-analyze/dashboards/create-dashboard

Create and manage rules
https://www.elastic.co/docs/explore-analyze/alerting/alerts/create-manage-rules

Elasticsearch query rule
https://www.elastic.co/docs/explore-analyze/alerting/alerts/rule-type-es-query

Metric threshold rule
https://www.elastic.co/docs/solutions/observability/incident-management/create-metric-threshold-rule

Custom threshold rule
https://www.elastic.co/docs/solutions/observability/incident-management/create-custom-threshold-rule

Kibana alerting prerequisites
https://www.elastic.co/docs/explore-analyze/alerting/alerts/alerting-setup

Infrastructure metric indices
https://www.elastic.co/docs/solutions/observability/infra-and-hosts/configure-settings

System integration and fields
https://www.elastic.co/docs/reference/integrations/system

Metricbeat stack monitoring
https://www.elastic.co/docs/deploy-manage/monitor/stack-monitoring/collecting-monitoring-data-with-metricbeat

Student takeaway

A successful observability workflow has four layers: collect trustworthy source data, validate raw documents, visualize with correctly formatted panels, and alert with appropriate thresholds, evaluation windows, and recovery/noise handling.